LeetCode Patterns Cheat Sheet

Contents

1 Sliding Window

Used for contiguous subarray/substring problems where a window expands and contracts over the input while maintaining some invariant (sum, count, distinct characters, etc.).

1.1 Fixed-size window

Time: $O(n)$ **Space:** $O(1)$ or $O(k)$

```
def max_sum_fixed_window(nums, k):
    window_sum = sum(nums[:k])
    best = window_sum
    for i in range(k, len(nums)):
        window_sum += nums[i] - nums[i - k]
        best = max(best, window_sum)
    return best
```

1.2 Variable-size window (two pointers, same direction)

Time: $O(n)$ **Space:** $O(1)$ or $O(\Sigma)$ for a character/count map

```
def smallest_subarray_with_sum_at_least(nums, target):
    left = 0
    total = 0
    best = float('inf')
    for right, val in enumerate(nums):
        total += val
        while total >= target:
            best = min(best, right - left + 1)
            total -= nums[left]
            left += 1
    return best if best != float('inf') else 0
```

Recognize it when: you need the longest/shortest/count of contiguous subarrays or substrings satisfying a condition, and elements can be added/removed incrementally.

Classic problems: Longest Substring Without Repeating Characters (3), Minimum Window Substring (76), Longest Repeating Character Replacement (424), Permutation in String (567), Max Consecutive Ones III (1004), Fruit Into Baskets (904), Subarrays with K Different Integers (992).

2 Two Pointers

Two indices moving through a (usually sorted) structure, either converging from both ends or moving in the same direction at different speeds.

2.1 Opposite ends (sorted array)

Time: $O(n)$ **Space:** $O(1)$

```
def two_sum_sorted(nums, target):
    left, right = 0, len(nums) - 1
    while left < right:
        s = nums[left] + nums[right]
        if s == target:
            return [left, right]
        elif s < target:
            left += 1
        else:
            right -= 1
    return [-1, -1]
```

Classic problems: Two Sum II (167), 3Sum (15), Container With Most Water (11), Trapping Rain Water (42), Valid Palindrome (125), Sort Colors / Dutch National Flag (75).

3 Fast & Slow Pointers (Floyd's Cycle Detection)

One pointer moves twice as fast as the other; useful for linked lists and detecting cycles or finding midpoints.

Time: $O(n)$ **Space:** $O(1)$

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            return True
    return False
```

Classic problems: Linked List Cycle (141), Linked List Cycle II (142), Middle of the Linked List (876), Happy Number (202), Find the Duplicate Number (287).

4 Prefix Sum & Difference Arrays

Precompute cumulative sums so range-sum queries become $O(1)$; a hash map of prefix sums turns "subarray sum equals k" problems into lookups.

Time: $O(n)$ preprocessing, $O(1)$ per query **Space:** $O(n)$

```
def subarray_sum_equals_k(nums, k):
    count = 0
    running = 0
    seen = {0: 1}
```

```
    for num in nums:
        running += num
        count += seen.get(running - k, 0)
        seen[running] = seen.get(running, 0) + 1
    return count
```

Classic problems: Subarray Sum Equals K (560), Range Sum Query - Immutable (303), Product of Array Except Self (238), Continuous Subarray Sum (523), Corporate Flight Bookings (1109, difference array).

5 Binary Search

Not just for sorted arrays – also for "search on the answer" when the answer space is monotonic (a feasibility check that flips from false to true).

5.1 Standard binary search

Time: $O(\log n)$ **Space:** $O(1)$

```
def binary_search(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

5.2 Binary search on the answer

Time: $O(n \log(\text{range}))$ **Space:** $O(1)$

```
def min_capacity(weights, days):
    def feasible(capacity):
        needed_days, load = 1, 0
        for w in weights:
            if load + w > capacity:
                needed_days += 1
                load = 0
            load += w
        return needed_days <= days

    lo, hi = max(weights), sum(weights)
    while lo < hi:
        mid = (lo + hi) // 2
        if feasible(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo
```

Recognize it when: the problem asks for a minimum/maximum value that satisfies some monotonic "is this good enough?" predicate.

Classic problems: Search in Rotated Sorted Array (33), Koko Eating Bananas (875), Capacity To Ship Packages Within D Days (1011), Split Array Largest Sum (410), Median of Two Sorted Arrays (4).

6 BFS / DFS on Graphs and Trees

Traversal patterns for exploring reachability, shortest paths (unweighted), connected components, and tree structure.

6.1 BFS (shortest path, level order)

Time: $O(V + E)$ **Space:** $O(V)$

```
from collections import deque

def bfs_shortest_path(graph, start, target):
    queue = deque([(start, 0)])
    visited = {start}
    while queue:
        node, dist = queue.popleft()
        if node == target:
            return dist
        for nxt in graph[node]:
            if nxt not in visited:
                visited.add(nxt)
                queue.append((nxt, dist + 1))
    return -1
```

6.2 DFS (connected components, backtracking traversal)

Time: $O(V + E)$ **Space:** $O(V)$ recursion stack

```
def dfs(graph, node, visited):
    visited.add(node)
    for nxt in graph[node]:
        if nxt not in visited:
            dfs(graph, nxt, visited)
```

Classic problems: Number of Islands (200), Rotting Oranges (994), Word Ladder (127), Clone Graph (133), Course Schedule (207, also topological sort), Pacific Atlantic Water Flow (417).

7 Topological Sort

Ordering of nodes in a DAG so that every directed edge $u \to v$ has u before v. Implemented via Kahn's algorithm (BFS with in-degrees) or DFS with a post-order stack.

Time: $O(V + E)$ **Space:** $O(V + E)$

```
from collections import deque
```

```python
def topo_sort(num_nodes, edges):
    graph = [[] for _ in range(num_nodes)]
    indegree = [0] * num_nodes
    for u, v in edges:
        graph[u].append(v)
        indegree[v] += 1

    queue = deque(i for i in range(num_nodes) if indegree[i] == 0)
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for nxt in graph[node]:
            indegree[nxt] -= 1
            if indegree[nxt] == 0:
                queue.append(nxt)
    return order if len(order) == num_nodes else []  # empty => cycle
```

Classic problems: Course Schedule (207), Course Schedule II (210), Alien Dictionary (269), Minimum Height Trees (310).

8 Backtracking

Explore a decision tree, trying candidates and undoing (*backtracking*) when a partial solution can't lead to a valid one. Used for permutations/combinations, constraint satisfaction, and exhaustive search with pruning.

Time: Exponential in general, pruning reduces it in practice **Space:** O(depth)

```python
def subsets(nums):
    result = []
    path = []

    def backtrack(start):
        result.append(path[:])
        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i + 1)
            path.pop()

    backtrack(0)
    return result
```

Classic problems: Subsets (78), Permutations (46), Combination Sum (39), N-Queens (51), Word Search (79), Palindrome Partitioning (131).

9 Dynamic Programming

Break a problem into overlapping subproblems with optimal substructure; cache results (memoization or bottom-up table) to avoid recomputation.

9.1 1D DP

Time: $O(n)$ **Space:** $O(n)$, often reducible to $O(1)$

```
def house_robber(nums):
    prev, curr = 0, 0
    for num in nums:
        prev, curr = curr, max(curr, prev + num)
    return curr
```

9.2 2D DP (grid / two sequences)

Time: $O(mn)$ **Space:** $O(mn)$, often reducible to $O(n)$

```
def longest_common_subsequence(text1, text2):
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i - 1] == text2[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]
```

9.3 Knapsack (0/1 and unbounded)

```
def knapsack_01(weights, values, capacity):
    dp = [0] * (capacity + 1)
    for w, v in zip(weights, values):
        for cap in range(capacity, w - 1, -1):  # reverse => each item
            once
            dp[cap] = max(dp[cap], dp[cap - w] + v)
    return dp[capacity]
```

Recognize it when: you're asked for a min/max/count over choices with recursive structure, and brute force overlaps subproblems (Fibonacci style, or "choose to include/exclude").

Classic problems: Climbing Stairs (70), House Robber (198), Coin Change (322), Longest Increasing Subsequence (300), Edit Distance (72), Partition Equal Subset Sum (416), Longest Common Subsequence (1143).

10 Greedy

Make the locally optimal choice at each step, provable to yield a globally optimal result (usually via an exchange argument or matroid structure). Often paired with sorting.

Time: $O(n \log n)$ (dominated by sort) **Space:** $O(1)$ or $O(n)$

```
def can_attend_all_meetings(intervals):
    intervals.sort(key=lambda x: x[0])
    for i in range(1, len(intervals)):
        if intervals[i][0] < intervals[i - 1][1]:
            return False
```

```
    return True
```

Classic problems: Jump Game (55), Gas Station (134), Task Scheduler (621), Non-overlapping Intervals (435), Minimum Number of Arrows to Burst Balloons (452).

11 Merge Intervals

Sort intervals by start time, then merge or compare adjacent ones. A specialization of the greedy pattern that shows up often enough to deserve its own bucket.

Time: $O(n \log n)$ **Space:** $O(n)$

```
def merge_intervals(intervals):
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
    for start, end in intervals[1:]:
        if start <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return merged
```

Classic problems: Merge Intervals (56), Insert Interval (57), Meeting Rooms II (253), Interval List Intersections (986).

12 Monotonic Stack

Maintain a stack that is always increasing or decreasing; used to find the next/previous greater or smaller element in $O(n)$ instead of $O(n^2)$.

Time: $O(n)$ **Space:** $O(n)$

```
def daily_temperatures(temps):
    answer = [0] * len(temps)
    stack = []  # indices, temps decreasing
    for i, t in enumerate(temps):
        while stack and temps[stack[-1]] < t:
            j = stack.pop()
            answer[j] = i - j
        stack.append(i)
    return answer
```

Classic problems: Daily Temperatures (739), Next Greater Element I/II (496, 503), Largest Rectangle in Histogram (84), Trapping Rain Water (42, stack variant), Remove K Digits (402).

13 Monotonic Deque (Sliding Window Maximum)

Combine sliding window with a monotonic deque to get window max/min in amortized $O(1)$ per step.

Time: $O(n)$ **Space:** $O(k)$

```
from collections import deque
```

```
def sliding_window_maximum(nums, k):
    dq = deque()  # indices, values decreasing
    result = []
    for i, num in enumerate(nums):
        while dq and nums[dq[-1]] < num:
            dq.pop()
        dq.append(i)
        if dq[0] <= i - k:
            dq.popleft()
        if i >= k - 1:
            result.append(nums[dq[0]])
    return result
```

Classic problems: Sliding Window Maximum (239), Shortest Subarray with Sum at Least K (862).

14 Heap / Top-K Elements

A (min/max) heap keeps the k best or worst elements seen so far without sorting everything, and merges multiple sorted structures efficiently.

Time: $O(n \log k)$ **Space:** $O(k)$

```
import heapq

def top_k_frequent(nums, k):
    counts = {}
    for n in nums:
        counts[n] = counts.get(n, 0) + 1
    heap = []
    for num, freq in counts.items():
        heapq.heappush(heap, (freq, num))
        if len(heap) > k:
            heapq.heappop(heap)
    return [num for freq, num in heap]
```

Classic problems: Kth Largest Element in an Array (215), Top K Frequent Elements (347), Merge k Sorted Lists (23), Find Median from Data Stream (295), Task Scheduler (621).

15 Union-Find (Disjoint Set)

Track connected components under union operations, with path compression and union by rank/size for near-$O(1)$ amortized operations.

Time: $O(\alpha(n))$ per operation **Space:** $O(n)$

```
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, x):
        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x])
```

```
        return self.parent[x]

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.rank[ra] < self.rank[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        if self.rank[ra] == self.rank[rb]:
            self.rank[ra] += 1
        return True
```

Classic problems: Number of Connected Components (323), Redundant Connection (684), Accounts Merge (721), Number of Islands II (305).

16 Trie (Prefix Tree)

Tree structure for storing strings by shared prefixes, enabling fast prefix lookups and autocomplete-style queries.

Time: $O(L)$ per insert/search, L = word length **Space:** $O(\Sigma \cdot N \cdot L)$

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_word = False

class Trie:
    def __init__(self):
        self.root = TrieNode()

    def insert(self, word):
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        node.is_word = True

    def search(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                return False
            node = node.children[ch]
        return node.is_word
```

Classic problems: Implement Trie (208), Word Search II (212), Design Add and Search Words Data Structure (211), Replace Words (648).

17 Bit Manipulation

XOR/AND/OR tricks for finding unique elements, counting bits, and subset/state compression (bitmask DP).

```python
def single_number(nums):
    result = 0
    for n in nums:
        result ^= n
    return result

def count_bits(n):
    return bin(n).count('1')
```

Classic problems: Single Number (136), Counting Bits (338), Number of 1 Bits (191), Sum of Two Integers (371), Subsets via bitmask (78 revisited).

18 Choosing a Pattern – Quick Reference

- **Contiguous subarray/substring, expand & contract:** Sliding Window.
- **Sorted array, pair/triplet sums:** Two Pointers.
- **Linked list cycle or midpoint:** Fast & Slow Pointers.
- **Range sum queries, "subarray sums to k":** Prefix Sum.
- **Sorted search space, or "minimize/maximize feasible value":** Binary Search (possibly on the answer).
- **Grid/graph reachability, shortest path unweighted:** BFS/DFS.
- **Ordering with dependencies:** Topological Sort.
- **All combinations/permutations, constraint search:** Backtracking.
- **Optimal value with overlapping subproblems:** Dynamic Programming.
- **Sort then make locally optimal choice:** Greedy / Merge Intervals.
- **Next greater/smaller element:** Monotonic Stack.
- **Window max/min:** Monotonic Deque.
- **Kth largest/smallest, frequency ranking:** Heap.
- **Group connectivity, cycle detection in undirected graph:** Union-Find.
- **Prefix-based string lookups:** Trie.
- **Uniqueness, subset enumeration, small state space:** Bit Manipulation.